

Internationalisation

Java Advanced Developer

StayAhead Training; Aug, 2025

About Internationalisation

Internationalisation

- The building of an app that adapts to various regions seamlessly
- The same executable works worldwide (no recompilation)
- Labels, messages, etc. not hard-coded
- The app is localised at runtime
- Also known as i18n (18 letters between the first and last letters)

Locale

Internationalisation

- An object that represents a geographical, political, or cultural region
- Commonly composed of language and country (region)
- Language on its own may be sufficient
- Used to construct formatters, e.g. `NumberFormat`

Locale: Examples

Internationalisation

Locale	Language	Country (region)
de-DE	German	Germany
de-AT	German	Austria
en-GB	English	Great Britain
en-AU	English	Australia

Locale: Obtaining a Locale

Internationalisation

```
import java.util.Locale;
```

```
...
```

```
var locale1 = Locale.getDefault();
```

```
var locale2 = Locale.forLanguageTag("fr-FR");
```

```
var locale3 = Locale.FRANCE;
```

```
var locale4 = Locale.FRENCH; // language only!
```

Locale: Obtaining a Locale

Internationalisation

```
import java.util.Locale;
```

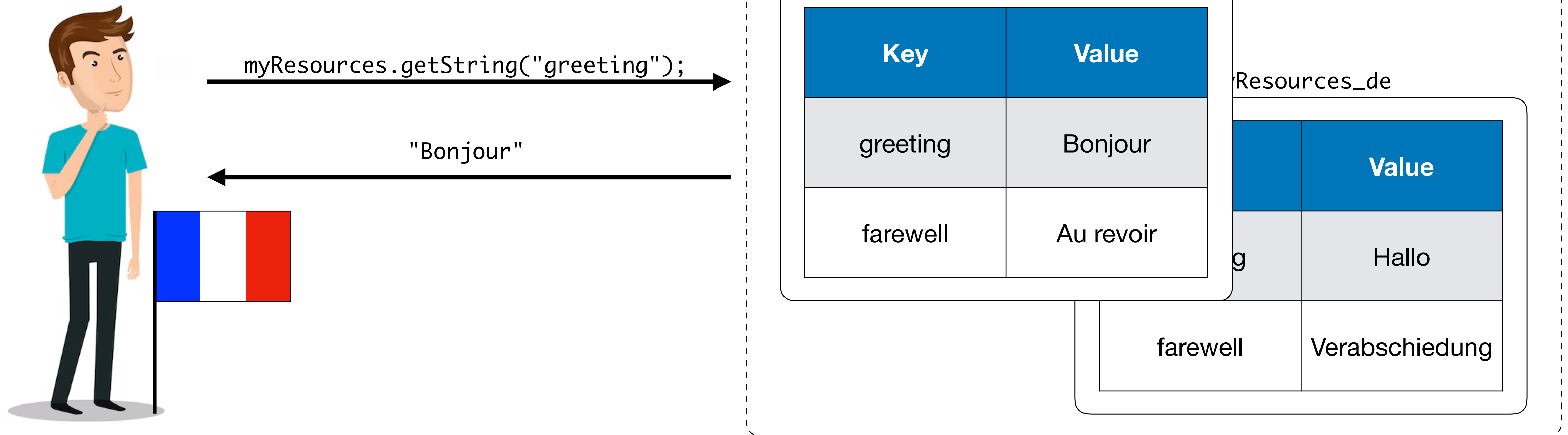
```
...
```

```
var locale5 = new Locale.Builder()  
    .setLanguage("fr")  
    .setRegion("FR")  
    .build();
```


ResourceBundle

Internationalisation

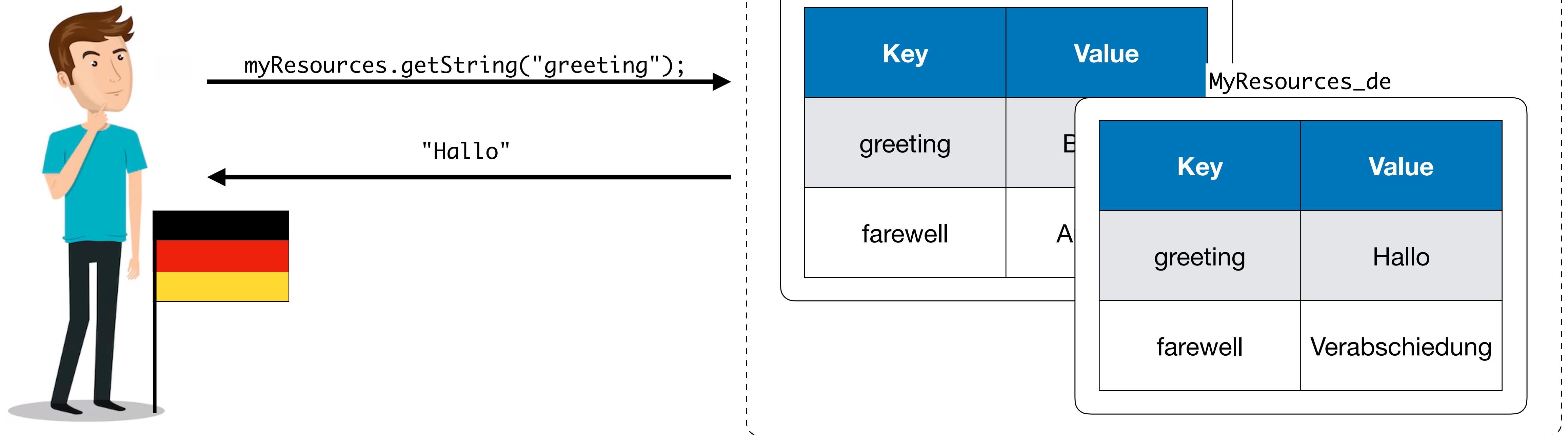
- An object comprising locale-specific key value pairs



ResourceBundle

Internationalisation

- An object comprising locale-specific key value pairs



ResourceBundle: List-type

Internationalisation

MyResources = base name

fr = Locale language

```
class MyResources_fr extends ListResourceBundle {
    @Override
    protected Object[][] getContents() {
        return new Object[][] {
            {"greeting", "Bonjour"},
            {"farewell", "Au revoir"}
        }
    }
}
```

ResourceBundle: List-type

Internationalisation

MyResources = base name

de = Locale language

```
class MyResources_de extends ListResourceBundle {
    @Override
    protected Object[][] getContents() {
        return new Object[][] {
            {"greeting", "Hallo"},
            {"farewell", "Verabschiedung"}
        }
    }
}
```

ResourceBundle: List-type

Internationalisation

```
var myResources = ResourceBundle.getBundle(  
    "com.example.i18n.MyResources", Locale.FRENCH);
```

```
var greeting = myResources.getString("greeting");  
assert greeting.equals("Bonjour");
```


ResourceBundle: Property-type

Internationalisation

my-resources = base name

fr = Locale language

```
# in a classpath file named my-resources_fr.properties  
greeting=Bonjour  
farewell=Au revoir
```

```
# in a classpath file named my-resources_de.properties  
greeting=Hallo  
farewell=Verabschiedung
```

ResourceBundle: Property-type

Internationalisation

```
var myResources = ResourceBundle.getBundle(  
    "my-resources", Locale.GERMAN);  
  
var greeting = myResources.getString("greeting");  
assert greeting.equals("Hallo");
```

Date Formatting

Internationalisation

- `java.text.DateFormat` to format `java.util.Date`
- `java.time.DateTimeFormatter` to format `java.time.LocalDate`
- Both may be used to format dates with time
- Both provide for a range of formats

Date Formatting: DateFormat

Internationalisation

```
import java.text.DateFormat;  
  
var date = new Date();  
  
var dateFormat = DateFormat.getDateInstance(  
    DateFormat.FULL,  
    Locale.forLanguageTag("fr-CA")  
);  
  
var localisedDate = dateFormat.format(date);
```

Date Formatting: DateTimeFormatter

Internationalisation

```
import java.time.DateTimeFormatter;

var date = LocalDate.now();

var dateTimeFormatter = DateTimeFormatter.ofLocalizedDate(
    FormatStyle.FULL,
);

var localisedDate = dateTimeFormatter
    .withLocale(Locale.forLanguageTag("fr-CA"))
    .format(date);
```


Number Formatting

Internationalisation

- `java.text.NumberFormat` for number, percentage, & currency formatting
- `java.text.DecimalFormat` for pattern-able* decimal number formatting
- *Primarily leading and trailing zeroes
- `DecimalFormat` typically created by casting `NumberFormat`

Number Formatting: Numbers

Internationalisation

```
var number = 123456.789;
```

```
var numberFormat = NumberFormat.getNumberInstance(  
    Locale.forLanguageTag("fr-FR"));
```

```
numberFormat.setMaximumFractionDigits(2);  
numberFormat.setRoundingMode(RoundingMode.DOWN);
```

```
assert numberFormat.format(number).equals("123 456,78");
```

Number Formatting: Percentages

Internationalisation

```
var percentage = 0.125; // assumed to be n/100
```

```
var numberFormat = NumberFormat.getPercentInstance(  
    Locale.forLanguageTag("fr-FR"));
```

```
numberFormat.setMaximumFractionDigits(1);
```

```
assert numberFormat.format(percentage).equals("12,5 %");
```


Number Formatting: Money

Internationalisation

```
var price = 19.95;
```

```
var numberFormat = NumberFormat.getCurrencyInstance(  
    Locale.forLanguageTag("fr-FR"));
```

```
assert numberFormat.format(price).equals("19,95 €");
```

Number Formatting: Decimals

Internationalisation

```
var number = 1.7793;
```

```
var numberFormat = NumberFormat.getNumberInstance(  
    Locale.forLanguageTag("fr-FR"));
```

```
var decimalFormat = (DecimalFormat) numberFormat;
```

```
decimalFormat.applyPattern("00.0#");
```

```
assert decimalFormat.format(number).equals("01,78");
```

Message Formatting

Internationalisation

- `java.text.MessageFormat` for `String` formatting
- Similar to `String.format` but:
 - uses argument indexes (not type specifiers)
 - provides implicit support for `Locale`
- See also `java.text.ChoiceFormat`

Message Formatting

Internationalisation

```
var messageFormat = new MessageFormat(  
    "Today will be {0} with a high temperature of {1}°C");  
  
var args = new Object[] {"sunny", 25};  
  
assert messageFormat.format(args).equals(  
    "Today will be sunny with a high temperature of 25°C");
```

Message Formatting: Type, Style, & Locale

Internationalisation

```
var messageFormat = new MessageFormat(  
    "The price for your service is {0, number, currency}",  
    Locale.forLanguageTag("en-US"));
```

number = type
currency = style

```
var args = new Object[] {375.50};
```

```
assert messageFormat.format(args).equals(  
    "The price for your service is $375.50");
```